ADL Exam — Predicted Question Bank

Sep 19, 2026 · @Trasha

Built from five lecture decks, the SSL and Lecture-2 notes, the diffusion preliminaries, and the 2025 and 2026 quizzes. The 2026 rubric is the strongest signal: every objective question carries 0.5 for the option and 0.5 for mechanism-level reasoning, so each question below comes with the reasoning that earns the second half-mark.

How the paper is marked

Half of every objective mark is the reasoning, and the 2026 key is explicit that narrative answers which do not engage with the mechanism earn nothing.

Case	Mark
Correct option + correct explanation	1.0
Correct option, no reasoning, or reasoning that contradicts the option	0.5
Correct reasoning, wrong option ticked	−0.25
Wrong option	−0.25
Unattempted	0

What this means for how you write:

- Give the mechanism, not the story. "Self-attention is permutation equivariant" is a story; "$Q' = PXW_Q = PQ$, so $E' = PEP^T$, and $P^TP = I$ cancels inside AV" is the mechanism.
- Carry dimensions through every step of a shape question. The 2026 key awarded the explanation half-mark for writing the shapes of K and V and both products, not for the final answer.
- Never tick an option your reasoning argues against — that combination scores −0.25, worse than the option alone.
- Negative marking means a genuinely blank guess costs 0.25 against 0 for leaving it. Attempt anything you can narrow to two options; leave only what you cannot reason about at all.
- The 2025 paper shows the subjective half: two or three sentences that name the mechanism and the trade-off. It accepted both the scaled and unscaled softmax in the cross-attention numerical, so state the convention you are using.

One observation worth acting on: the 2026 quiz drew all five questions from attention, ViT and SSL, and none from video or diffusion. The 2025 quiz added CKA and factorised-attention complexity. A one-hour paper covering the full syllabus will almost certainly reach into video and the diffusion derivations that the quizzes skipped.

Part A — Attention and transformers

The 2026 quiz took three of five questions from here, so expect this block to be the densest part of the paper.

A1. Degenerate queries. *(Asked verbatim on a Lecture-3 slide — treat it as pre-announced.)* In a self-attention layer all query vectors are identical, while keys and values differ. What does the layer compute, and what is lost?

Answer. Every row of E is identical, so after softmax every position shares one attention distribution a, and $Y_i = \Sigma_j a_j V_j$ is the same vector for every i. The layer collapses to a single content-weighted global pooling of V, replicated across positions. What is lost is all position-specific selectivity: no token can query for what *it* needs, so the output carries no per-token information.

A2. Degenerate keys. Same layer, but now all keys are identical while queries differ. What happens?

Answer. $E_{i,j} = Q_i \cdot k / \sqrt{D_Q}$ is constant along j within each row, and softmax of a constant row is uniform, 1/N. So $Y_i = \text{mean}(V)$ for every i. Note the contrast with A1: identical queries still give *content-selective* weights (they depend on the keys); identical keys give *uniform* weights, i.e. pure mean pooling with no selection at all. That contrast is the explanation mark.

A3. Permutation. *(This was 2026 Q2 and a Lecture-2 slide, so a variant is very likely again.)* Input tokens are permuted, $X' = PX$, with no positional encoding. How do E and Y transform?

Answer. $E' = PEP^T$ and $Y' = PY$. Row-wise linear maps commute with row permutation, so $Q' = PQ$, $K' = PK$, $V' = PV$. Then $E' = Q'K'^T = PQK^TP^T = PEP^T$, row-wise softmax commutes with this, and $Y' = A'V' = PAP^TPV = PAV = PY$ using $P^TP = I$. Self-attention is permutation *equivariant*, not invariant — a pooled or [CLS] readout on top of it is what becomes invariant.

A4. Why divide by $\sqrt{D_Q}$? Short subjective.

Answer. If query and key components are roughly independent with unit variance, the dot product of D_Q-dimensional vectors has variance D_Q, so logits grow like $\sqrt{D_Q}$. Large-magnitude logits drive softmax into saturation, where one weight goes to 1 and the

Jacobian of softmax goes to zero — gradients vanish and training stalls. Dividing by √D_Q restores unit-scale logits. Say "variance of the dot product grows with dimension"; "to normalise" alone will not earn the mark.

A5. Masking mechanism. In masked self-attention, why are the forbidden entries set to −∞ rather than 0?

Answer. The mask is applied to E *before* softmax. exp(−∞) = 0, so those positions receive exactly zero weight and the remaining weights still sum to 1. Zeroing *after* softmax would leave the row un-normalised, and zeroing E entries before softmax would give exp(0) = 1, i.e. a large weight rather than no weight.

A6. Cross-attention. Where do Q, K and V come from in the decoder's cross-attention layer, and what classical mechanism does it generalise?

Answer. Q comes from the previous decoder layer; K and V both come from the encoder output. It is the transformer's version of Bahdanau attention in seq2seq: the decoder state queries the encoder's hidden states so each output position can emphasise the relevant part of the input. Self-attention draws all three from the same sequence; cross-attention draws Q from one sequence and K, V from another.

A7. The seq2seq bottleneck. Why did attention have to be added to encoder–decoder RNNs?

Answer. Vanilla seq2seq compresses the whole input into a single fixed-size context vector c (often h_T), so capacity does not grow with T and long inputs are lost. Attention computes a fresh c_t = Σ_i a_{t,i} h_i at every decoder step, so the decoder reads a different weighted view of *all* encoder states. The weights are learned end-to-end by backprop, never supervised.

A8. FFN's role. Attention already mixes information across the sequence. Why keep a position-wise feed-forward network?

Answer. They do complementary jobs. Self-attention mixes *across* tokens but is linear in V given the weights; the FFN processes each position *independently* and non-linearly, refining that position's own aggregated representation. Calling the block "token mixing then channel mixing" is exactly the framing that makes MLP-Mixer fall out (Part B).

A9. Positional encoding. Write the sinusoidal scheme and state one property that makes it attractive.

```
\mathrm{PE}(pos,\,2i) = \sin\!\left(\frac{pos}{10000^{2i/p}}\right), \qquad
\mathrm{PE}(pos,\,2i+1) = \cos\!\left(\frac{pos}{10000^{2i/p}}\right)
```

Answer. Even dimensions take sine, odd take cosine, i runs 0 to p/2 − 1, and p is the model dimension. Wavelengths form a geometric progression from about 2π to 10000·2π, so different dimensions resolve position at different scales. It needs no learned parameters and extends to sequence lengths longer than those seen in training, which a learned lookup table cannot. The encoding is *added* to the embedding, not concatenated.

A10. BERT vs GPT. One line each on architecture, objective and directionality.

Answer. BERT stacks transformer *encoder* blocks, is trained on masked language modelling plus next-sentence prediction, and is bidirectional — every token conditions on both left and right context. GPT stacks transformer *decoder* blocks with masked multi-head self-attention, is trained on next-token prediction, and is left-to-right only. GPT has no encoder, hence no cross-attention module. BERT-Base: 12 layers, hidden 768, 12 heads, 110M parameters; BERT-Large: 24 layers, hidden 1024, 16 heads, 340M.

A11. The [CLS] token. Why can a token with no content of its own summarise a sentence?

Answer. It is prepended and then passes through the same self-attention stack as every other token, so it aggregates contextual information from the whole sequence. Because nothing else is tied to it, fine-tuning is free to shape its final hidden state into whatever sentence-level representation the task head needs. This is also why removing positional embeddings makes a ViT's [CLS] output invariant to patch order — 2026 Q3.

A12. Vanishing gradients. Why do deep RNNs suffer, and how does attention sidestep it?

Answer. BPTT differentiates through a long composition of non-linear maps, so the gradient is a product of many Jacobians; that product tends to shrink or blow up exponentially in sequence length. Attention gives a direct, constant-length path from any decoder step to any encoder state, so the gradient does not have to traverse T recurrent steps.

A13. Complexity. State the time and memory cost of self-attention over N tokens with dimension d, and name the term that dominates for images.

Answer. The score matrix is N × N, so both time and memory are $O(N^2 d)$ and $O(N^2)$ respectively. For an R × R image treated as pixels, $N = R^2$ and the attention matrix has R^4 entries — the deck's worked case is R = 128 with 48 layers and 16 heads reaching about 768 GB for one example, which is why patches replaced pixels.

A14. Multi-head. Why H heads of dimension d/H rather than one head of dimension d?

Answer. One softmax produces one attention distribution per query, so a single head can only attend to one pattern at a time; H heads run in parallel on independent projections and their outputs are concatenated, letting the layer attend to several relationships simultaneously. Splitting the dimension keeps the parameter and FLOP count roughly unchanged.

Part B — ViT, Swin, MLP-Mixer, distillation

B1. Why patches. Four ideas were tried in order: attention inside a CNN, local attention replacing convolution, a transformer on raw pixels, a transformer on patches. Why did only the fourth work?

Answer. Idea 1 leaves the model a CNN. Idea 2 (Stand-Alone Self-Attention, Local Relation Nets) has many fiddly details, is hard to implement and only marginally beats ResNets. Idea 3 is killed by memory: an $R \times R$ image gives R^4 attention entries. Patches cut the token count by the square of the patch size — 224×224 with 16×16 patches gives 196 tokens and a $197^2 \approx 38.8$k-entry attention matrix, about 150 KB, which is tractable.

B2. Data scaling. ViT loses to ResNets on ImageNet, matches them with ImageNet-21k pretraining, and beats them with JFT-300M. Explain.

Answer. Convolution hard-codes locality, translation equivariance and a hierarchical scale structure. ViT has almost none of this — only the patch grid — so those regularities must be learned from data. With little data that missing prior is a handicap; with enough data the model learns better-suited features than the hand-imposed prior allows. The deck also notes ViT trained in 2.5k TPU-v3 core days against ResNet's 9.9k, because matrix multiply maps to hardware better than convolution.

B3. Hierarchical vs isotropic. Contrast ViT and ResNet layouts and say why it matters downstream.

Answer. ResNets decrease resolution and increase channels with depth ($56^2 \to 28^2 \to 14^2$ at $64 \to 128 \to 256$ channels) — hierarchical, matching the fact that objects appear at many scales. ViT keeps every block at the same resolution and width (e.g. $768 \times 14 \times 14$) — isotropic. Dense tasks such as detection and segmentation want multi-scale feature maps, which is precisely the gap Swin fills.

B4. Swin patch merging. Walk through one merge step with shapes.

Answer. Start from 4×4 patches projected to C channels, giving $C \times H/4 \times W/4$. Concatenate each 2×2 neighbourhood: $4C \times H/8 \times W/8$. Linearly project $4C \to 2C$ (a 1×1 convolution): $2C \times H/8 \times W/8$. Repeating gives $4C$ at $H/16$ and $8C$ at $H/32$. For Swin-T, $C = 96$, so the final stage is $7 \times 7 \times 768$, then global average pooling to $1 \times 1 \times 768$.

B5. Window attention complexity. *(2025 Q5 asked exactly this.)* Give the cost of full attention and of $M \times M$ window attention on an $H \times W$ token grid.

Answer. Full attention needs $(HW)^2 = H^2W^2$ entries — quadratic in image area. Windows give M^4 entries per window $\times$ $(H/M)(W/M)$ windows $= M^2HW$, i.e. **linear** in image size for fixed M. Swin uses $M = 7$ throughout. Worked case: 224×224 with a 56×56 grid gives $56^4 \approx 9.8$M entries for full attention.

B6. Shifted windows. *(Lecture-4 slide MCQ.)* Replace every shifted-window block with ordinary non-shifted window attention, everything else unchanged. Most likely consequence?

- (A) Accuracy is unaffected
- (B) Tokens interact normally inside each window, but cross-window communication is substantially restricted
- (C) The model becomes equivalent to a standard ViT with global attention
- (D) Patch merging can no longer reduce spatial resolution

Answer: (B). With fixed partitions, a token's receptive field never crosses its window boundary within a stage; information only leaks across boundaries through patch merging, which is slow. Alternating normal and shifted windows in successive blocks lets every token reach neighbours in the adjacent window at the next layer. (C) is wrong because windows stay local, (D) because merging is independent of the attention pattern.

B7. Relative position bias. How does Swin encode position, and why not ViT's scheme?

```
A = \mathrm{Softmax}\!\left(\frac{QK^{\top}}{\sqrt{D}} + B\right)V, \qquad
Q,K,V \in \mathbb{R}^{M^2 \times D}, \; B \in \mathbb{R}^{M^2 \times M^2}
```

Answer. Swin drops absolute positional embeddings and adds a learned bias B indexed by the *relative* offset between two patches inside a window. Attention within a window should depend on how far apart two patches are, not where the window sits in the image; a relative scheme is also reusable across windows and across input resolutions. The ablation table reports rel. pos. beating abs. pos. and no-position.

B8. MLP-Mixer. What replaces self-attention, and what is the architectural cost?

Answer. Viewing a transformer block as "mix across tokens, then mix across channels", Mixer keeps the channel-mixing MLP (C → C, applied to each of N patches) and replaces attention with a token-mixing MLP (N → N, applied to each of C channels). The cost is that the token-mixing matrix has a fixed size N, so the mixing pattern is learned once and is identical for every input — no content-dependent routing — and the model cannot accept a different token count. ImageNet results were unremarkable; JFT pretraining helped. Like ViT, it is isotropic with no hierarchy.

B9. Distillation basics. Why does a softened teacher distribution beat a one-hot target? *(Lecture slide: "suppose the teacher predicts almost exactly one-hot probabilities — would that make it an excellent teacher?")*

Answer. No — that is the worst case. The value of a teacher is the relative mass it puts on the *wrong* classes, which encodes similarity structure between classes. A one-hot teacher collapses distillation into ordinary cross-entropy against a label, giving the

student nothing beyond what the ground truth already supplies. Temperature scaling exists precisely to expose that structure and to stop the teacher being overconfident.

B10. A weaker teacher. Teacher A has 85% ImageNet accuracy, teacher B has 83%. Can B be the better teacher for a ViT student?

Answer. Yes. Top-1 accuracy measures only the argmax; what transfers is the full distribution and the inductive bias behind it. A CNN teacher with slightly lower accuracy can carry locality and translation-equivariance priors that a ViT lacks, and may be better calibrated. DeiT's teacher is RegNetY-16GF (84M parameters), a CNN, for exactly this reason.

B11. Where the inductive bias comes from. Model A: CNN, weak augmentation. Model B: ViT, strong augmentation. Model C: ViT, strong augmentation plus CNN teacher. Locate the visual inductive bias in each.

Answer. A: in the architecture — convolution's locality, weight sharing and hierarchy. B: in the data — augmentations define the invariances (a model trained with RandAugment and MixUp is *told* which transformations preserve the label). C: additionally in the supervision signal — the CNN teacher's outputs carry its bias to the student even though the student's architecture is unchanged. This is the defensible reading of "distillation helps"; it is *not* evidence that transformers learn convolutional filters, nor that distillation adds independent training samples.

B12. Hard vs soft distillation in DeiT. State the objective and one reason hard distillation is used.

```
\mathcal{L}_{\mathrm{global}} = \tfrac{1}
{2}\,\mathcal{L}_{\mathrm{CE}}\big(\psi(Z_s),\, y\big) + \tfrac{1}
{2}\,\mathcal{L}_{\mathrm{CE}}\big(\psi(Z_s),\, y_t\big), \qquad y_t =
\arg\max_c Z_t(c)
```

Answer. Soft distillation matches the teacher's full probability vector (KL, with temperature); hard distillation uses the teacher's argmax as a label, with label smoothing $\varepsilon = 0.1$. The deck's argument for hard targets: augmentations such as MixUp *change the true label* — two mixed images have a mixed label — so the ground-truth label attached to an augmented crop can be wrong, while the teacher's prediction on that same augmented image is a coherent target. DeiT also adds a **distillation token** alongside the class token: the class token is supervised by the ground truth, the distillation token by the teacher, and both are used at test time.

B13. MixUp's premise. MixUp trains on $x' = \lambda x_i + (1-\lambda)x_j$ with $y' = \lambda y_i + (1-\lambda)y_j$. Why should $f(\lambda x_1 + (1-\lambda)x_2) \approx \lambda f(x_1) + (1-\lambda)f(x_2)$?

Answer. It is a deliberately imposed prior, not a fact about images. Requiring approximate linearity between training points forces the model to vary smoothly off the data manifold instead of producing sharp, overconfident regions there. That shrinks the effective hypothesis class, softens decision boundaries and improves calibration — which is why it helps a low-inductive-bias model like ViT more than a CNN.

B14. Design question. *(A Lecture-4 slide poses this; it is the most likely long subjective.)* Distil a ResNet trained on ImageNet down to CIFAR-10 in a 10-way 5-shot setting, then deploy on an edge device. Training data is scarce and the device needs a small model.

Answer. Two separate problems, solved in two stages. **Adaptation under 50 labelled images:** do not fine-tune the whole ResNet — it will overfit. Freeze the backbone and train only a lightweight head, or use parameter-efficient adaptation (adapters, LoRA, BatchNorm-only tuning). A metric-learning head fits few-shot naturally: compute class prototypes as the mean embedding of the 5 support images and classify by nearest prototype, trained episodically to match the 10-way 5-shot test condition. **Edge deployment:** distil the adapted ResNet into a small student. Distillation does not need labels, so you can run the teacher over unlabelled CIFAR-10 or heavily augmented copies and train the student on the teacher's soft outputs — this manufactures the supervision that the 50 labels cannot provide. Finish with quantisation and pruning. The key insight to state: the label scarcity is solved by freezing plus episodic training, the size constraint by distillation on unlabelled data, and the soft targets are what let a small student learn from far more inputs than there are labels.

Part C — Self-supervised learning

C1. Four definitions. Distinguish unsupervised, self-supervised and semi-supervised learning in one line each.

Answer. *Unsupervised*: the model is not told what to predict at all (clustering, density estimation) — older terminology. *Self-supervised*: the model is trained to predict a signal that occurs naturally in the raw data (the next pixel, the missing patch, the rotation angle) rather than a human annotation. *Semi-supervised*: joint training on a little labelled data and a lot of unlabelled data. The motivating arithmetic: 1M images at 10 s each and $15/hour is about $41.7k; 1B images is about $41.7M.

C2. Pretext then transfer. Draw the two-stage recipe and state the goal.

Answer. Stage 1: train encoder φ and decoder ψ on a pretext task with loss $L(y, y')$, no labels. Stage 2: discard ψ, keep φ, and transfer to downstream tasks (classification, detection, segmentation) via a linear probe, KNN, or fine-tuning. The goal is that pretrain-then-transfer beats both supervised pretraining and training directly on the downstream task.

C3. Taxonomy. Sort the pretext tasks into families.

Family	Tasks
Generative — predict part of the input	Autoencoders (sparse, denoising, masked), autoregressive models, GANs, colorization, inpainting
Discriminative — predict something about the input	Context prediction, jigsaw, rotation, clustering, contrastive
Multimodal — use a signal beyond RGB	Video, 3D/depth, sound, language

C4. Autoencoder bottleneck. In a plain autoencoder, what is the *only* thing preventing a trivial solution, and how do sparse and denoising variants change that?

Answer. Only H < D — the hidden layer being narrower than the input. With H ⩾ D the network can learn the identity and learn nothing. A sparse autoencoder replaces the width constraint with an activation penalty, $L(x) = \|x - x'\|^2_2 + \lambda\|h\|_1$, so the code may be wide but must be mostly zero. A denoising autoencoder corrupts the input (randomly zeroing pixels) and asks for the *clean* image, so the identity is no longer available and the model must use context. MAE is the denoising autoencoder idea with a ViT and patch-level masking.

C5. Shuffle sensitivity. *(This was 2026 Q5 — expect a relative.)* Model A's pretext accuracy barely changes when image patches are shuffled; Model B's drops sharply. Which transfers better to object recognition?

Answer: Model B. If shuffling does not hurt, the pretext task is solvable from a bag of patches — local texture and colour statistics alone — so the encoder is never required to represent how parts are arranged. Model B's task (jigsaw, context prediction, masked-patch modelling) *needs* spatial layout, forcing part-to-whole structure, which is what recognition depends on. Robustness to shuffling is a shortcut, not a strength. Note the rubric's wording: because the question says "most reasonable", a good counter-example may earn credit at the instructor's discretion.

C6. Augmentation defines invariance. *(2026 Q4.)* An augmentation consistently removes a visual attribute from one view of a positive pair. Effect on the representation?

Answer. The encoder becomes invariant to that attribute, whether or not it is useful downstream. The contrastive loss pulls the two views together; if one view has the attribute and the other does not, the only way to minimise the loss is to stop encoding it. So augmentations *define* the invariances of the encoder, and information an augmentation

destroys is discarded regardless of downstream value — colour jitter removes colour, which then hurts flower classification. "Same source image" and "large batch" do not preserve information.

C7. Exemplar CNN. What is the task, and what breaks?

Answer. Take N patches, sample K augmentations of each, and train an N-way classifier to predict which original image an augmented patch came from — an explicit invariance-to-augmentation objective. It breaks because the final layer has parameters proportional to N, so it cannot scale to large datasets. Contrastive learning is the fix: replace the parametric classifier with similarity in embedding space, so nothing grows with dataset size.

C8. Contrastive loss. Write the pair loss and the InfoNCE loss, and say what τ does.

```
L_S(x_1,x_2) = d^2, \qquad L_D(x_1,x_2) = \max(0,\, m-d)^2, \qquad d = \lVert
\phi(x_1)-\phi(x_2)\rVert_2
```

```
s(i,j) = \frac{\phi(x_i)^{\top}\phi(x_j)}
{\lVert\phi(x_i)\rVert\,\lVert\phi(x_j)\rVert}, \qquad L_i = -\log
\frac{\exp\!\big(s(i,j)/\tau\big)}{\sum_{k=1,\,k\neq i}^{2N}
\exp\!\big(s(i,k)/\tau\big)}
```

Answer. The margin form pulls positives together and pushes negatives apart *only up to margin m* — beyond m they contribute no gradient, which stops the loss wasting capacity on already-separated pairs. InfoNCE turns this into a (2N−1)-way classification: with a batch of N images and two augmentations each, every view must identify its partner among the other 2N−1. Low τ sharpens the distribution and concentrates gradient on the hardest negatives; high τ makes it uniform. Negatives come for free from the rest of the batch, which is why SimCLR needs large batches and MoCo instead keeps a momentum-updated queue.

C9. Linear probe vs fine-tuning. *(2025 Q3a.)* MIM pretraining gives mediocre linear-probe numbers but excellent fine-tuning numbers. Explain.

Answer. A linear probe can only read off directions that are already linearly separable by class. MIM optimises pixel reconstruction, so it learns local, texture-level features that are informative but not linearly class-separable; fine-tuning adapts them and adds the global structure the task needs. Contrastive learning, by contrast, explicitly separates image-level embeddings, so its features are linearly separable almost by construction and it wins linear probing. The general point to state: **a linear probe measures linear separability, not information content** — the two protocols answer different questions.

C10. Joint MIM + contrastive. *(2025 Q3b.)* Would training both objectives together help?

Answer. Yes, if balanced. MIM captures local structure and high-frequency detail; CL enforces global semantic separation. L = (1−λ)L_MIM + λL_CL gives more transferable features than either alone, and the ICLR-23 study finds even the simplest harmonisation leverages both. The caveat: the objectives pull in different directions (invariance vs reconstruction), so an unbalanced λ can degrade both.

C11. MAE mechanics. Describe the pipeline and the asymmetry that makes it cheap.

Answer. Tokenise every patch by linear projection plus positional embedding; randomly shuffle the token list and drop the tail according to the masking ratio (equivalent to sampling without replacement); encode only the remaining visible patches with a ViT; append mask tokens, unshuffle to restore alignment with targets, add positional embeddings, and decode with a *small* ViT that predicts the pixel values of the masked patches. The asymmetry is the point: with a high masking ratio the encoder — the expensive part, and the part you keep — sees only a small fraction of tokens, so pretraining is cheap and scales to large ViTs. MAE pretraining beats training from scratch and beats MoCo-v3 on fine-tuning across ViT-B/L/H.

C12. The data rant. What is the methodological complaint about SSL evaluation?

Answer. SSL is motivated by scaling to data too large to label, yet most papers pretrain on *unlabelled ImageNet* and evaluate on ImageNet. Unlabelled ImageNet is still curated: one object per image, balanced classes. On genuinely uncurated data, SSL in vision has not scaled the way it did in NLP. A fair answer also notes that fair comparison is hard — architecture, pretraining dataset, probe layer, and transfer protocol all vary between papers, and reimplementations change the numbers.

C13. CLIP. State the objective and explain zero-shot classification.

Answer. Contrastive over 400M (image, text) pairs from the internet: in a batch, each image must identify its matching caption and each caption its matching image, over a matrix of similarities. Zero-shot classification then works because the classifier is written in language — embed each candidate class name as a text prompt, embed the image, and take the nearest text embedding. No task-specific training data is needed for a new label set. The stated drawback: CLIP's training set is private and cannot be reproduced (RedCaps, 12M pairs from 350 subreddits, is one open response).

C14. Why language as the extra signal? Three reasons.

Answer. *Semantic density* — a few words carry rich information. *Universality* — language can describe any concept, so the label space is open rather than a fixed taxonomy. *Scalability* — non-experts can caption images and paired data can be harvested from the web at scale. Compare with other multimodal signals: video gives temporal context, sound gives an audio track, 3D gives depth or point clouds.

Part D — MIM vs CL, and what self-supervised ViTs learn

D1. The vocabulary problem. BERT's masked language modelling predicts a token from a finite vocabulary with a softmax. What is the corresponding vocabulary for images, and how does BEiT construct it?

Answer. There isn't one — pixels are continuous, so there is no discrete set to classify over. BEiT manufactures one with a discrete VAE (dVAE, from DALL-E): the dVAE is trained to reconstruct the image conditioned on discrete latent codes, and those codes become "visual tokens". Pretraining then has **two views** of the same image — the patch view (used in pretraining *and* fine-tuning) and the visual-token view (used only to define the pretraining target). Masked patches are fed to the transformer, and the model predicts the correct visual token for each masked position. Visual tokens matter because they summarise detail into a higher-level abstraction, so the model is not forced to predict exact pixel values.

D2. Blockwise masking. Why mask a contiguous block of patches rather than scattered individual patches?

Answer. Scattered single patches can usually be inferred from their immediate neighbours by low-level interpolation, which is a shortcut that teaches the encoder nothing about object structure. Masking a whole block removes that local crutch and forces longer-range reasoning. The same logic returns in VideoMAE's tube masking (Part E).

D3. SimMIM's four axes. Name the four design dimensions and SimMIM's finding on each.

Axis	Finding
Masking strategy	Simple **random** masking works; moderately large masked patches (e.g. 32) with ratio 0.6 are effective
Encoder	Vanilla ViT *or* Swin — both work, so the recipe is not architecture-specific
Prediction head	A single **linear** layer performs no worse than a 2-layer MLP or an inverse Swin; heavy heads waste capacity in the head itself, which is discarded
Prediction target	Direct regression of **raw pixels** with $\ell 1$ or $\ell 2$ loss is enough; no dVAE tokenizer needed

The headline is that MIM does not need BEiT's machinery — random masking, a linear head and pixel regression suffice. Default setup: Swin-B, 192 × 192 input, AdamW with cosine schedule, 100 epochs, batch 2048, lr 8e-4, mask ratio 0.6.

D4. Masked-only vs whole-image loss. SimMIM computes the loss on the masked area only. What changes if you compute it over the whole image?

Answer. Loss on masked patches alone is a pure **prediction** task. Including visible patches makes it a joint **prediction and reconstruction** task, and the reconstruction part is nearly free — the model can copy visible input through — so gradient is spent on a trivial objective and performance drops. Stating "the unmasked term is trivially satisfiable" is the mechanism the marker wants.

D5. The central comparison. *(High-probability exam question.)* Summarise what CL and MIM each learn.

	Contrastive learning	Masked image modelling
Attention range	Long-range, global patterns	Mostly local relationships
Where it acts	Crucial in **later** layers	Mainly **early** layers
Frequency content	Low-frequency → **shape**-oriented	High-frequency → **texture**-oriented
Failure mode	Attention collapse into homogeneity	None of this kind
Wins at	Linear probing, small models	Fine-tuning, large models, dense prediction

D6. Attention collapse. What is it, how is it measured, and why does it cap CL's fine-tuning accuracy?

Answer. In CL-trained ViTs, representations become highly similar across heads, across depths, and across spatial coordinates — measured by cosine similarity along each of those three axes, which is higher for CL than MIM, especially in later layers. Homogeneous representations carry little extra information, so adding heads or depth does not help: a large part of the network's capacity is wasted. That is why MIM's fine-tuning accuracy exceeds CL's in large models. It also explains CL's weakness on dense prediction — if token features are homogeneous with respect to spatial coordinate, per-pixel tasks have nothing to work with.

D7. Why CL is linearly separable. Explain the token-geometry argument.

Answer. Take the ~196 spatial tokens of an image as a cloud of points. CL's self-attention translates all tokens of an image roughly equally, so distances *within* an image do not grow, but the *centres* of different images' clouds move apart — giving linearly separable image-level representations and strong linear probes. MIM's self-attention transforms

tokens differently depending on the query, so distances *within* an image increase: tokens stay diverse and spatially discriminative, which suits dense prediction but not a linear probe on a pooled vector.

D8. Harmonisation. Write the combined objective and note one practical instability.

```
L = (1-\lambda)\,L_{\mathrm{MIM}} + \lambda\,L_{\mathrm{CL}}
```

Answer. Even this simplest combination captures both advantages. A separate practical point from MoCo-v3: CL training of ViTs is unstable when the patch-embedding layer is learned — keeping it as a *fixed random projection* stabilises training. This also suggests positional embeddings are not the sole source of positional inductive bias.

D9. CKA. *(2025 Q1 was exactly this.)* Define centred kernel alignment and state why it is used instead of a regression-based comparison.

```
\mathrm{CKA}(X,Y) = \frac{\lVert Y^{\top}X\rVert_F^{2}}{\lVert
X^{\top}X\rVert_F \, \lVert Y^{\top}Y\rVert_F}
```

Answer. $X \in \mathbb{R}^{n\times p_1}$ and $Y \in \mathbb{R}^{n\times p_2}$ hold activations of two layers for the same n examples, column-centred. The key insight is to *not* compare features of an example directly across the two representations — they may have different widths and arbitrary bases. Instead, measure similarity between every **pair of examples** within each representation separately (the Gram matrices XX^T and YY^T), then compare those similarity *structures*. The identity $\mathrm{tr}(MM^T) = \|M\|^2_F$ gives the computable form above. CKA is invariant to orthogonal transformation and isotropic scaling — so a permuted or rotated basis scores the same — but not to arbitrary invertible linear maps, which is what makes it discriminative where CCA is not. It also handles $p_1 \neq p_2$, so ViT and ResNet layers are comparable.

D10. What CKA showed for ViT vs ResNet. Three findings.

Answer. (1) ViT's layer-similarity matrix is a uniform **grid-like** pattern with high similarity between lower and higher layers; ResNet shows **clear stages** with low similarity between its lower and higher layers. (2) A large number of lower ResNet layers correspond to a small set of the lowest ViT layers — ViT computes low-level features differently and in fewer layers. (3) ViT's highest layers have representations quite different from ResNet's. The interpretation: ViT propagates representations strongly from lower to higher layers, largely because of residual connections and uniform resolution.

D11. Mean attention distance. Define it and state what it reveals about data scale.

Answer. For each query token, average the pixel-space distance to every key token, weighted by that token's attention weight; average over tokens and heads to get a per-layer number — the analogue of a CNN's receptive field size. Finding: lower ViT layers

contain heads that attend **both locally and globally**, while higher layers are almost entirely global. Crucially, **when trained on less data the lower layers fail to learn local attention**, and those models perform much worse — evidence that local feature extraction, which CNNs hard-code, matters for strong performance and is the specific thing ViT must learn from data. Relatedly, ResNet effective receptive fields are highly local and grow gradually, whereas ViT's shift from local to global.

D12. BEiT's payoff. What did BEiT demonstrate about annotation efficiency?

Answer. BEiT-Large reached comparable ImageNet top-1 with roughly 224× less labelled data than ViT-Large trained from scratch — 14M instead of 3B images — and also accelerated fine-tuning convergence. A qualitative result worth citing: BEiT's self-attention maps separate objects without ever seeing a manual annotation.

Part E — Video

The quizzes have not touched video yet, which makes it likelier in a full paper.

E1. Fusion strategies. Compare late fusion, early fusion and 3D CNNs in one line each, with the failure of the first two.

Answer. *Late fusion*: run a 2D CNN on every frame independently, then pool or flatten and combine at the end — gets high-level appearance per frame but **cannot compare low-level motion between frames**, because frames never interact until after all spatial processing. *Early fusion*: reshape T frames into 3T channels so the first convolution sees all frames — compares frames immediately, but **one layer of temporal processing is not enough** and all temporal information is collapsed at once. *3D CNN (slow fusion)*: 3D convolution and pooling build up receptive field gradually in space **and** time. Note the baseline lesson: a single-frame 2D CNN is a very strong baseline (77.7 vs C3D's 84.4 on Sports-1M) — always try it first.

E2. Receptive field numerical. *(Directly from a Lecture slide; the arithmetic is examinable.)* Input 3 × 20 × 64 × 64. Give the output size and receptive field (T × H × W) at each layer for the three designs.

Layer	Late fusion	Early fusion	3D CNN
Conv (3×3 or 3×3×3)	12×20×64×64, RF 1×3×3	12×64×64, RF 20×3×3	12×20×64×64, RF 3×3×3
Pool (4×4 or 4×4×4)	12×20×16×16, RF 1×6×6	12×16×16, RF 20×6×6	12×5×16×16, RF 6×6×6
Conv (3×3 or 3×3×3)	24×20×16×16, RF 1×14×14	24×16×16, RF 20×14×14	24×5×16×16, RF 14×14×14

Layer	Late fusion	Early fusion	3D CNN
GlobalAvgPool	24, RF 20×64×64	24, RF 20×64×64	24, RF 20×64×64

The recurrence: RF_out = RF_in + (k − 1) · jump_in, and jump_out = jump_in · stride. So 3 → 3 + 3·1 = 6 after a stride-4 pool, then 6 + 2·4 = 14. Read the pattern off the table: late fusion builds **slowly in space, all-at-once in time at the end**; early fusion is **all-at-once in time at the start**; the 3D CNN builds slowly in both.

E3. 2D vs 3D convolution. With input C_in × T × H × W, contrast the kernel shapes and outputs, and say why it matters.

Answer. Early fusion's 2D kernel is C_out × C_in × T × 3 × 3 and slides over x and y only, producing C_out × H × W — a 2D grid; the temporal axis is consumed by the kernel, so the filter has a *fixed* weighting for each time offset. The 3D kernel is C_out × C_in × 3 × 3 × 3 and slides over x, y **and** t, producing C_out × T × H × W — a 3D grid. The consequence the slide asks about: to detect, say, a blue-to-orange transition *anywhere* in space and time you need translation equivariance in time as well as space, which only the sliding 3D kernel provides.

E4. C3D's cost. Why is C3D expensive, with numbers?

Answer. Every layer is 3×3×3 convolution with 2×2×2 pooling (except Pool1, which is 1×2×2 to preserve time early). A 3×3×3 kernel has 3× the parameters and roughly 3× the FLOPs of a 3×3 kernel at the same channel count, and it is applied across T frames. Result: 39.5 GFLOP against VGG-16's 13.6 and AlexNet's 0.7 — about 2.9× VGG.

E5. Optical flow and two-stream. Define optical flow and explain why a separate stream helps.

Answer. Optical flow is a displacement field F between consecutive frames: F(x,y) = (dx,dy) with I_{t+1}(x+dx, y+dy) = I_t(x,y). It isolates local motion from appearance. The two-stream network runs a spatial stream on a single RGB image (3 × H × W) and a temporal stream by early fusion on a stack of flow fields (2(T−1) × H × W), then fuses the predictions. It helps because motion is explicitly computed rather than implicitly learned, so the network need not discover it from raw pixels. UCF-101: spatial-only 73, temporal-only 83.7, average fusion 86.9, SVM fusion 88 — note that the *temporal* stream alone beats the spatial stream.

E6. Long-term structure. Why add an LSTM on top of a CNN, and what does each contribute?

Answer. 3D CNNs model only local motion over ~2–5 second clips. Inside a CNN, each value is a function of a *fixed* temporal window — local temporal structure; inside an RNN, each vector is a function of *all* previous vectors — global temporal structure. Stacking a

recurrent net on CNN features gives both. In practice the CNN is often frozen as a feature extractor and gradients are not propagated into it, to save memory.

E7. Non-local block. Describe it and the initialisation trick.

Answer. Take a 3D CNN feature map C × T × H × W and treat it as a set of THW vectors of dimension C. Produce queries, keys and values with 1×1×1 convolutions, compute a (THW) × (THW) softmax attention matrix, take the weighted sum of values, project back, and **add a residual connection**. The trick: initialise the last convolution to zero, so the whole block initially computes the identity and can be dropped into an already-trained 3D CNN without disturbing it.

E8. I3D inflation. How are 3D weights bootstrapped from 2D, and why does the scaling factor matter?

Answer. Replace each K_h × K_w convolution/pooling layer with a K_t × K_h × K_w version, and initialise the 3D kernel by **copying the 2D kernel K_t times along time and dividing by K_t**. The division matters because it makes the inflated network produce *exactly* the same output as the 2D network on a constant video (the same frame repeated K_t times) — so the model starts from the pretrained image solution instead of from noise. On Kinetics-400 this takes a two-stream inflated CNN from 62.2 trained from scratch to 74.2 with ImageNet pretraining.

E9. ViViT tokenisation. Contrast uniform frame sampling with tubelet embedding.

Answer. *Uniform frame sampling*: sample n_t frames, embed each independently exactly as ViT does, giving n_t · n_h · n_w tokens — temporal information is fused only later, by attention. *Tubelet embedding*: extract non-overlapping spatio-temporal tubes of size t × h × w and project each linearly, giving n_t = ⌊T/t⌋, n_h = ⌊H/h⌋. This **fuses spatio-temporal information during tokenisation**. Smaller tubelets mean more tokens and more computation. Notation: ViViT-B/16×2 is a ViT-Base backbone with a 16 × 16 × 2 tubelet.

E10. ViViT's four models. Name them and give the complexity argument for Model 2.

Answer. Model 1 — joint spatio-temporal attention over all tokens. Model 2 — factorised encoder: a spatial encoder over tokens within each frame, then a temporal encoder over the resulting per-frame representations. Model 3 — factorised self-attention: two MSA modules inside each block, spatial then temporal. Model 4 — factorised dot-product attention: one Q, but half the heads attend over spatial keys/values (K_s, V_s) and half over temporal (K_t, V_t), then concatenate.

```
\text{Model 1: } O\big((n_t \cdot n_h \cdot n_w)^2\big) \qquad\text{vs}\qquad
\text{Model 2: } O\big(n_t (n_h n_w)^2 + n_t^2\big)
```

Model 2 has *more* layers and parameters than Model 1 but *fewer* FLOPs — a distinction worth stating explicitly, since it is the kind of trap an MCQ is built on.

E11. Transferring an image ViT to video. Three initialisation problems and their fixes.

Answer. Video datasets are orders of magnitude smaller than image datasets, so video models are initialised from pretrained image models. (1) *Positional embeddings*: the video model has n_t × more tokens, so the image embeddings are **repeated temporally** — at initialisation all tokens sharing a spatial index have the same embedding, then fine-tuning differentiates them. (2) *Tubelet embedding filter*: E is 3D whereas E_image is 2D, so either inflate by replicating along time and averaging, E = [E_image,…,E_image]/t, or use **central-frame initialisation**, E = [0,…,E_image,…,0], zero everywhere except the centre ⌊t/2⌋. (3) *Model 3's extra MSA*: initialise the spatial MSA from the pretrained module and **all weights of the temporal MSA to zero**, so the block starts out equivalent to the pretrained image block.

E12. VideoMAE and tube masking. *(A Lecture slide poses this as an MCQ.)* Suppose VideoMAE used independently sampled random masks in every frame instead of masking the same spatial region across consecutive frames. Which shortcut becomes more likely?

- (A) The model infers a missing region in one frame by finding nearly the same visible content in an adjacent frame
- (B) The model loses all positional information
- (C) The decoder cannot reconstruct RGB pixels
- (D) Temporal attention becomes equivalent to spatial attention

Answer: (A). Consecutive video frames are highly redundant — the same content is usually visible a frame away with only small displacement. With independent per-frame masks, a masked patch almost always has a visible near-copy in a neighbouring frame, so reconstruction reduces to temporal copying and no semantic understanding is learned. **Tube masking** masks the same spatial region through all sampled frames, removing that leak, which is what permits the extreme 90–95% masking ratio. Same principle as BEiT's blockwise masking (D2). Supporting detail: temporal sampling with stride τ = 4 on Kinetics and 2 on Something-Something, cube embedding treats each 2 × 16 × 16 cube as one token giving T/2 × H/16 × W/16 tokens, and the backbone is a vanilla ViT with joint space-time attention.

Part F — VAE ELBO and diffusion

Notation follows your uploaded preliminaries: the forward step is q(x_t | x_{t−1}) = N(x_t; √α_t x_{t−1}, (1−α_t)I), so α_t = 1 − β_t in DDPM's β notation, and ᾱ_t = Π_{s=1}^{t} α_s.

F1. Derive the VAE ELBO by Jensen's inequality.

$$\log p_\theta(x) = \log \int p_\theta(x,z)\,dz = \log \int q_\phi(z\mid x)\,\frac{p_\theta(x,z)}{q_\phi(z\mid x)}\,dz = \log \mathbb{E}_{q_\phi(z\mid x)}\!\left[\frac{p_\theta(x,z)}{q_\phi(z\mid x)}\right]$$

$$\geq \mathbb{E}_{q_\phi(z\mid x)}\!\left[\log \frac{p_\theta(x,z)}{q_\phi(z\mid x)}\right] = \underbrace{\mathbb{E}_{q_\phi(z\mid x)}\big[\log p_\theta(x\mid z)\big]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}\big(q_\phi(z\mid x)\,\|\,p(z)\big)}_{\text{regularisation}}$$

Steps to show. Multiply and divide by q_φ(z|x) to turn the intractable integral into an expectation. Apply Jensen's inequality to the concave log, which moves it inside. Split log p(x,z) = log p(x|z) + log p(z) and group the two log-ratio terms into a KL. Say what each term does: the first wants accurate reconstruction, the second pulls the approximate posterior towards the prior.

F2. Derive the ELBO the second way, and state what the gap is. *(Ask for this if the question says "show the bound is tight when …".)*

$$D_{\mathrm{KL}}\big(q_\phi(z\mid x)\,\|\,p_\theta(z\mid x)\big) = \mathbb{E}_{q_\phi}\big[\log q_\phi(z\mid x) - \log p_\theta(z\mid x)\big] = \mathbb{E}_{q_\phi}\big[\log q_\phi(z\mid x) - \log p_\theta(x,z)\big] + \log p_\theta(x)$$

$$\Longrightarrow\quad \log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi}\!\left[\log \frac{p_\theta(x,z)}{q_\phi(z\mid x)}\right]}_{\mathrm{ELBO}} + \underbrace{D_{\mathrm{KL}}\big(q_\phi(z\mid x)\,\|\,p_\theta(z\mid x)\big)}_{\geq 0}$$

Answer. Use log p_θ(z|x) = log p_θ(x,z) − log p_θ(x) and pull log p_θ(x) out of the expectation (it does not depend on z). This is an **equality**, which is stronger than the Jensen route: it shows the bound's slack is exactly the KL between the approximate and true posterior. The bound is tight iff q_φ(z|x) = p_θ(z|x). Maximising the ELBO therefore does two things at once — raises the likelihood and tightens the approximation.

F3. Reparameterisation. Why is it needed, and write the Gaussian KL in closed form.

Answer. The ELBO's first term is an expectation over q_φ, whose *parameters* are what we differentiate. $\nabla_\varphi E_{q_\varphi}[f(z)]$ cannot be written as E[∇_φ f(z)] because the sampling distribution itself depends on φ, and sampling is not differentiable. Write z = μ_φ(x) + σ_φ(x) ⊙ ε with ε ~ N(0, I): now the randomness sits in ε, which carries no parameters, the expectation is over a fixed distribution, and the gradient passes straight through μ and σ. The alternative score-function (REINFORCE) estimator is unbiased but has much higher variance.

```
D_{\mathrm{KL}}\big(\mathcal{N}(\mu,\sigma^2 I)\,\|\,\mathcal{N}(0,I)\big) = \tfrac{1}{2}\sum_{j=1}^{J}\left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\right)
```

F4. The conditional-probability identities. *(Straight from your preliminaries note; the valid/invalid drill is very examinable.)* For the chain $x_0 \to \cdots \to x_{t-1} \to x_t \to x_{t+1}$, mark each valid or invalid and give the reason.

Statement	Verdict	Reason
$q(x_t \mid x_{t-1}, x_0) = q(x_t \mid x_{t-1})$	Valid	Markov — once x_{t-1} is known, x_0 adds nothing about x_t
$q(x_{t+1} \mid x_t, x_{t-1}) = q(x_{t+1} \mid x_t)$	Valid	Same Markov property, one step along
$q(x_t \mid x_{t-1}, x_{t+1}) = q(x_t \mid x_{t-1})$	**Invalid**	Conditioning on the **future** is different: x_{t+1} was generated *from* x_t, so it carries information about x_t
$q(x_t, x_{t-1} \mid x_0) = q(x_t \mid x_0)\, q(x_{t-1} \mid x_t, x_0)$	Valid	joint = marginal × conditional
$q(x_t, x_{t-1} \mid x_0) = q(x_t \mid x_{t-1}, x_0)\, q(x_{t-1} \mid x_t, x_0)$	**Invalid**	A joint is marginal × conditional, never the product of two conditionals

The intuition to quote for row 3: past states before x_{t-1} are **screened off** by x_{t-1}, but a future observation x_{t+1} still carries information about x_t. The note's Gaussian example — $x_t = x_{t-1} + \varepsilon_t$, $x_{t+1} = x_t + \varepsilon_{t+1}$, $\varepsilon \sim \mathcal{N}(0,1)$: given $x_{t-1} = 0$, $x_t \sim \mathcal{N}(0,1)$; additionally observing $x_{t+1} = 6$ makes values of x_t near 6 far more likely. By Bayes, $q(x_t \mid x_{t-1}, x_{t+1}) \propto q(x_{t+1} \mid x_t)\, q(x_t \mid x_{t-1})$, and the extra factor $q(x_{t+1} \mid x_t)$ changes the posterior.

F5. Why x_0 appears in the reverse posterior. Explain the teacher–student reading.

Answer. In the *forward* direction x_0 is redundant, $q(x_t \mid x_{t-1}, x_0) = q(x_t \mid x_{t-1})$. In the *reverse* direction it is highly informative: $q(x_{t-1} \mid x_t, x_0)$ is a tractable Gaussian, whereas $q(x_{t-1} \mid x_t)$ alone is not. So $q(x_{t-1} \mid x_t, x_0)$ is the **ground-truth denoising posterior**, available during training because x_0 is the training image — the *teacher*. The learned reverse model $p_\theta(x_{t-1} \mid x_t)$ has no access to x_0 at generation time — the *student*. Training matches student to teacher; sampling runs the student alone.

F6. Derive the closed form of the forward process, $q(x_t \mid x_0)$.

```
x_t = \sqrt{\alpha_t}\,x_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_{t-1} =
\sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\,x_{t-2} + \sqrt{1-\alpha_{t-
1}}\,\epsilon_{t-2}\right) + \sqrt{1-\alpha_t}\,\epsilon_{t-1}
```

```
= \sqrt{\alpha_t \alpha_{t-1}}\,x_{t-2} + \underbrace{\sqrt{\alpha_t(1-
\alpha_{t-1})}\,\epsilon_{t-2} + \sqrt{1-\alpha_t}\,\epsilon_{t-
1}}_{\text{sum of two independent Gaussians}}
```

Key step. The two noise terms are independent zero-mean Gaussians, so their sum is Gaussian with variance $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$. Hence $x_t = \sqrt{(\alpha_t\alpha_{t-1})}\, x_{t-2} + \sqrt{(1-\alpha_t\alpha_{t-1})}\, \varepsilon$. By induction:

```
q(x_t \mid x_0) = \mathcal{N}\!\left(x_t;\ \sqrt{\bar{\alpha}_t}\,x_0,\ (1-
\bar{\alpha}_t)\,I\right), \qquad \bar{\alpha}_t = \prod_{s=1}^{t}\alpha_s
```

This is what lets training sample an arbitrary timestep in one shot instead of simulating t steps. Note $\bar{\alpha}_t \to 0$ as $t \to T$, so $q(x_T \mid x_0) \to \mathcal{N}(0, I)$ regardless of x_0.

F7. Derive the reverse posterior $q(x_{t-1} \mid x_t, x_0)$.

```
q(x_{t-1}\mid x_t, x_0) = \frac{q(x_t \mid x_{t-1}, x_0)\; q(x_{t-1}\mid
x_0)}{q(x_t \mid x_0)} = \frac{q(x_t \mid x_{t-1})\; q(x_{t-1}\mid x_0)}
{q(x_t \mid x_0)}
```

Answer. Bayes' rule conditioned throughout on x_0, then the Markov property kills x_0 in the first factor (row 1 of F4). All three factors are known Gaussians from F6, so expanding the exponents and completing the square in x_{t-1} gives a Gaussian:

```
q(x_{t-1}\mid x_t, x_0) = \mathcal{N}\big(x_{t-1};\ \tilde{\mu}_t(x_t,x_0),\
\tilde{\beta}_t I\big)
```

```
\tilde{\mu}_t(x_t,x_0) = \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-
\bar{\alpha}_t}\,x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,(1-\alpha_t)}{1-
\bar{\alpha}_t}\,x_0, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-
\bar{\alpha}_t}\,(1-\alpha_t)
```

Sanity checks worth stating: the mean is a convex-style blend of the noisy observation x_t and the clean target x_0, and $\tilde{\beta}_t < 1-\alpha_t$, i.e. knowing x_0 reduces the variance below the forward noise level.

F8. Derive the diffusion ELBO and name its three terms.

```
\log p_\theta(x_0) \geq \mathbb{E}_q\!\left[\log \frac{p_\theta(x_{0:T})}
{q(x_{1:T}\mid x_0)}\right] = \mathbb{E}_q\!\left[\log
\frac{p(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1}\mid x_t)}{\prod_{t=1}^{T} q(x_t
\mid x_{t-1})}\right]
```

Rewrite q(x_t | x_{t−1}) = q(x_{t−1} | x_t, x_0) q(x_t | x_0) / q(x_{t−1} | x_0) for t ⩾ 2 — this is the crucial move, since it replaces a forward transition with the tractable reverse posterior. The product telescopes, and the bound regroups into:

```
-\mathcal{L} = \underbrace{D_{\mathrm{KL}}\big(q(x_T\mid
x_0)\,\|\,p(x_T)\big)}_{L_T:\ \text{prior matching}} + \sum_{t=2}^{T}
\underbrace{D_{\mathrm{KL}}\big(q(x_{t-1}\mid x_t,x_0)\,\|\,p_\theta(x_{t-
1}\mid x_t)\big)}_{L_{t-1}:\ \text{denoising matching}} \;\underbrace{-\ \log
p_\theta(x_0\mid x_1)}_{L_0:\ \text{reconstruction}}
```

What to say about each. L_T has **no learnable parameters** — the forward process is fixed and ā_T ≈ 0 makes it near zero. L_{t−1} is the whole job: a KL between two Gaussians, which has a closed form, so training needs no sampling of the reverse chain. L_0 is the final decoding step back to pixels. Compare with the VAE ELBO: diffusion is a hierarchical VAE whose encoder is **fixed, not learned**, and whose latents have the same dimension as the data.

F9. From the KL to the simple loss. Show how L_{t−1} becomes noise prediction.

Answer. Both distributions are Gaussian with the same (fixed) variance, so the KL reduces to a scaled squared difference of means, ‖μ̃_t(x_t,x_0) − μ_θ(x_t,t)‖² / (2β̃_t). Invert F6, x_0 = (x_t − √(1−ā_t) ε)/√ā_t, substitute into μ̃_t, and parameterise μ_θ in the same form. The x_t terms cancel and what remains is a difference of noise terms, giving after dropping the weighting:

```
L_{\mathrm{simple}} = \mathbb{E}_{t,\,x_0,\,\epsilon}\left[\left\lVert
\epsilon - \epsilon_\theta\!\left(\sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-
\bar{\alpha}_t}\,\epsilon,\ t\right)\right\rVert^2\right]
```

So the network predicts the noise that was added, the timestep t is an input, and training is: sample x_0, sample t uniformly, sample ε, form x_t in one step, regress ε.

F10. Likely short-answer probes on this material.

- *Why can't we just use q(x_{t−1} | x_t) directly?* It requires marginalising over the whole data distribution and is intractable; conditioning on x_0 makes it Gaussian, and that is only possible during training.
- *Why is the forward process fixed rather than learned?* It removes the encoder from the optimisation, makes L_T parameter-free, and guarantees a known N(0,I) endpoint to sample from.

- *Why are diffusion latents the same size as the image, unlike a VAE's bottleneck?* There is no compression objective; the "code" is a noisy image, so the reverse model can be a U-Net or ViT operating in pixel space.
- *What breaks if α_t is too close to 1 for all t?* The forward process barely adds noise, so x_T is not approximately $N(0,I)$, L_T is large, and sampling from the prior starts far off the learned chain. Too small, and x_0 is destroyed early and the denoising steps carry no signal.
- *Where does Jensen's inequality enter the diffusion ELBO?* The same place as in the VAE: log of an expectation over $q(x_{1:T}|x_0)$ becomes an expectation of a log.

Part G — Numerical drills

Both quizzes opened with a numerical. Work these with a pen; the explanation half-mark is awarded for carrying dimensions through, not for the final number.

G1. Attention shapes. $Q \in \mathbb{R}^{8\times32}$, $X \in \mathbb{R}^{12\times64}$, $W_K \in \mathbb{R}^{64\times32}$, $W_V \in \mathbb{R}^{64\times16}$. Give the shapes of K, V, E, A and Y.

Answer. $K = XW_K \in \mathbb{R}^{12\times32}$; $V = XW_V \in \mathbb{R}^{12\times16}$; $E = QK^T$ is $(8\times32)(32\times12) = \mathbb{R}^{8\times12}$ — one row per query, one column per key; $A = \text{softmax}(E) \in \mathbb{R}^{8\times12}$; $Y = AV$ is $(8\times12)(12\times16) = \mathbb{R}^{8\times16}$. **Rules to state:** the output has one row per *query* and width D_V; queries and keys must share D_Q for the dot product; N_Q and N_X never need to match (that is what makes cross-attention possible).

G2. Self-attention shapes. Now $X \in \mathbb{R}^{N\times D}$ with $W_Q, W_K \in \mathbb{R}^{D\times d_k}$ and $W_V \in \mathbb{R}^{D\times d_v}$.

Answer. $Q, K \in \mathbb{R}^{N\times d_k}$, $V \in \mathbb{R}^{N\times d_v}$, E and $A \in \mathbb{R}^{N\times N}$ (square, because every token is both query and key), $Y \in \mathbb{R}^{N\times d_v}$. In a real transformer block $d_v = D$ so the residual connection can be added.

G3. Softmax by hand. $Q = [1\ 1\ 0]$, K rows $k_1 = [1\ 0\ 0]$, $k_2 = [0\ 1\ 0]$, $k_3 = [1\ 1\ 0]$, $V = [2; 0; 0]$, $d_k = 3$. Compute the output both unscaled and scaled.

Answer. $S = QK^T = [1, 1, 2]$.

Unscaled: $\text{softmax}([1,1,2]) = [e, e, e^2]/(2e + e^2) = [1, 1, e]/(2 + e) \approx [0.2120, 0.2120, 0.5760]$. Output $= 2(0.2120) =$ **0.4239**.

Scaled by √3: $S/\sqrt{3} = [0.5774, 0.5774, 1.1547]$, exponentials $[1.7814, 1.7814, 3.1732]$, sum 6.7360, weights $\approx [0.2645, 0.2645, 0.4711]$. Output $= 2(0.2645) =$ **0.5290**.

The 2025 key accepted either convention, so **state which one you used**. Dividing e and e^2 through by e to get $[1, 1, e]/(2+e)$ is the trick that keeps the arithmetic clean without a calculator.

G4. The 2025 cross-attention question, worked. Q = [[1,0,0],[0,0,1]], K = [[0,0,1],[1,1,1],[1,0,1]], V = [[1],[0],[0]].

Answer. $S = QK^T$: row 1 is $q_1 \cdot k_i$ = [0, 1, 1]; row 2 is [1, 1, 1]. Softmax row 1 = [1, e, e]/(1+2e); row 2 is uniform [1/3, 1/3, 1/3] because the row is constant. Output = AV picks the first column of V, giving [1/(1+2e); 1/3] ≈ [**0.1554**; **0.3333**]. Row 2 is the degenerate case from A2 — constant scores give mean pooling.

G5. ViT token count and memory. 224 × 224 image, 16 × 16 patches.

Answer. 224/16 = 14, so a 14 × 14 grid = 196 patches, plus the [CLS] token = **197 tokens**. Attention matrix per head: $197^2 \approx 38{,}809$ entries (the deck rounds to $14^4 = 38{,}416$ ignoring [CLS]), about 150 KB in float32. With 48 layers × 16 heads that is about 112 MB for one example. With 14 × 14 patches instead (a 16 × 16 grid) it is $256^2 = 65{,}536$ entries, 256 KB, and about 192 MB total. Contrast with pixels: 128 × 128 at the same depth needs roughly 768 GB.

G6. ViT-L/16 parameters. Reproduce the decomposition, and note the discrepancy.

Component	Count
Patch embedding, 16×16×3×1024	786,432
Positional embedding, 1024×197	201,728
Transformer blocks, 24 × (1024²1024·3 + 1024·4096·2)	276,824,064
Classification head, 1024×1000	1,024,000

Configuration: patch 16, embedding 1024, 24 blocks, 16 heads, MLP hidden 4096, 1000 classes.

Worth knowing before the exam. The slide quotes the block total as ≈337,002,496 and the grand total as ≈339M, but 24 × (3,145,728 + 8,388,608) = 276,824,064, not 337M — the printed formula and the printed total do not agree. The formula as written counts only W_Q, W_K, W_V and the two MLP matrices; adding the output projection W_O (1024^2 = 1,048,576 per block) gives 24 × 12,582,912 = 301,989,888 and a grand total of ≈**304M**, which is the published ViT-L/16 figure. Safest approach in the exam: write the decomposition line by line and name what you are counting (QKV, W_O, two MLP matrices, biases and LayerNorm ignored). The marks are in the decomposition.

G7. Sanity-check BERT-Base. Verify the 110M figure from d = 768, 12 layers, 12 heads, FFN 3072, vocabulary 30,522.

Answer. Per block: attention $4 \times 768^2 = 2{,}359{,}296$ (W_Q, W_K, W_V, W_O); FFN $2 \times 768 \times 3072 = 4{,}718{,}592$; total ≈ 7,077,888. Twelve blocks ≈ 84.9M. Token embeddings $30{,}522 \times 768$ ≈ 23.4M. Sum ≈ 108M, i.e. the quoted 110M once biases, LayerNorm, position and segment embeddings are included. Learn this method — it generalises to any "how many parameters" question.

G8. Swin vs full attention, with numbers. 224 × 224 image, 4 × 4 patches, M = 7.

Answer. Grid is 56 × 56 = 3136 tokens. Full attention: $3136^2 = 56^4 =$ **9,834,496** entries. Window attention: $(56/7)^2 = 64$ windows, each $49 \times 49 = 2401$, total $64 \times 2401 =$ **153,664** = $M^2HW = 49 \times 3136$. That is a **64×** reduction, and the saving grows with image size because one is quadratic and the other linear.

G9. ViViT Model 1 vs Model 2. 32 frames, 224 × 224, tubelet 16 × 16 × 2.

Answer. $n_t = \lfloor 32/2 \rfloor = 16$, $n_h = n_w = \lfloor 224/16 \rfloor = 14$, so $16 \times 14 \times 14 =$ **3136 tokens**. Model 1 (joint attention): $3136^2 \approx$ **9.83M** attention entries. Model 2 (factorised encoder): $n_t(n_h n_w)^2 + n_t^2 = 16 \times 196^2 + 16^2 = 614{,}656 + 256 =$ **614,912**, about **16×** cheaper — despite having *more* layers and parameters.

G10. VideoMAE token budget. 16 frames, 224 × 224, cube 2 × 16 × 16, mask ratio 90% and 95%.

Answer. $T/2 \times H/16 \times W/16 = 8 \times 14 \times 14 =$ **1568 tokens**. At 90% masking the encoder processes about 157 tokens; at 95%, about 78. That is the entire point of the asymmetric design — the encoder's attention cost falls by roughly 100× and 400× respectively, since cost is quadratic in token count.

G11. Swin patch merging shapes. Swin-T with C = 96 on a 224 × 224 input. Give the four stage shapes.

Answer. Stage 1: 96 × 56 × 56. Stage 2: 192 × 28 × 28. Stage 3: 384 × 14 × 14. Stage 4: 768 × 7 × 7. Each merge concatenates a 2 × 2 neighbourhood (4C channels) then linearly projects 4C → 2C, so resolution halves and width doubles. Global average pooling over the final 7 × 7 gives a 768-dimensional vector.

G12. Positional-encoding arithmetic. For p = 512, which dimension has the longest wavelength, and roughly what is it?

Answer. Wavelength is $2\pi \cdot 10000^{2i/p}$. i = 0 gives the shortest, 2π; $i = p/2 - 1 = 255$ gives $2\pi \cdot 10000^{510/512} \approx 2\pi \cdot 10000$, about 62,832. So low dimensions oscillate fast and resolve fine position, high dimensions oscillate slowly and resolve coarse position — a multi-scale positional code, which is why the geometric progression is chosen.

What to prioritise, and the facts to have ready

Priority order

1. **Questions posed on the slides themselves.** The 2026 quiz lifted its permutation question straight from a Lecture-2 slide and its shifted-window logic from a Lecture-4 slide. Every in-deck question is reproduced above: A1, A2, A3, B6, B9, B10, B11, B14, C5, D4, E2, E12. Treat these as pre-announced.
2. **Attention mechanics with shapes.** Both quizzes opened here. G1–G4.
3. **The MIM vs CL comparison table.** D5 — the single densest source of one-line objective questions in the syllabus.
4. **The VAE and diffusion derivations.** You flagged these, they have never been quizzed, and derivations are the natural fit for a subjective section. F1, F2, F6, F7, F8.
5. **Complexity arguments.** Swin's M^2HW, ViViT Model 1 vs 2, the R^4 pixel-attention blow-up. Asked in 2025 Q5.
6. **Video.** Untouched by both quizzes, and a full paper has room for it.

Formula sheet

$$E = \frac{QK^{\top}}{\sqrt{D_Q}},\quad A = \mathrm{softmax}(E),\quad Y = AV \qquad L_i = -\log\frac{\exp(s_{i,j}/\tau)}{\sum_{k\neq i}\exp(s_{i,k}/\tau)}$$

$$\mathrm{ELBO} = \mathbb{E}_{q_\phi}\big[\log p_\theta(x\mid z)\big] - D_{\mathrm{KL}}\big(q_\phi(z\mid x)\,\|\,p(z)\big), \qquad \log p_\theta(x) = \mathrm{ELBO} + D_{\mathrm{KL}}\big(q_\phi(z\mid x)\,\|\,p_\theta(z\mid x)\big)$$

$$q(x_t\mid x_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}x_0,\,(1-\bar\alpha_t)I\big), \qquad \tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}(1-\alpha_t), \qquad \mathrm{CKA} = \frac{\lVert Y^{\top}X\rVert_F^2}{\lVert X^{\top}X\rVert_F\lVert Y^{\top}Y\rVert_F}$$

Numbers worth memorising

Quantity	Value
ViT tokens, 224^2 with 16^2 patches	196 + 1 [CLS] = 197
Swin window size M	7, throughout the network
Swin-T channel width C	96 → 192 → 384 → 768

Quantity	Value
Full vs window attention, 56×56 grid	9.8M vs 153,664 entries
SimMIM mask ratio / VideoMAE mask ratio	0.6 / 0.90–0.95
VideoMAE cube	2 × 16 × 16
BERT-Base / BERT-Large	12L-768-12H-110M / 24L-1024-16H-340M
Original transformer	6 encoder layers, FFN 512, 8 heads
C3D cost	39.5 GFLOP vs VGG-16's 13.6
Two-stream on UCF-101	spatial 73, temporal 83.7, fused 86.9–88
Labelling cost	~$41.7k per 1M images
CLIP pretraining scale	400M image–text pairs
BEiT label efficiency	224× less labelled data (3B → 14M)

Five one-line answers that recur

- **Permutation:** self-attention is equivariant, not invariant; a fixed [CLS] row makes the readout invariant.
- **Inductive bias:** ViT lacks locality and hierarchy, so it must learn them from data — hence the data-scale crossover, and hence why a CNN teacher helps.
- **Augmentation:** augmentations define the encoder's invariances; whatever they destroy is discarded regardless of downstream usefulness.
- **Linear probe vs fine-tune:** a probe measures linear separability, not information — CL wins probes, MIM wins fine-tuning.
- **Masking:** any masking scheme that leaves a near-copy visible (scattered patches, per-frame random masks in video) creates a copying shortcut; blockwise and tube masking remove it.

One gap to close

You mentioned separate VAE and diffusion material with the full derivation worked through. That file did not come through — the only diffusion document uploaded is the conditional-probability preliminaries, which stops before the ELBO itself. Part F above reconstructs the standard derivation in your notes' α_t notation and matches its teacher–student framing, but if your slides define the loss weighting differently, or derive $\tilde{\mu}_t$ by a different route, send them and I will align Part F to them.